1. **STUDI KASUS : Pecahkan Misteri 2 Process Bersama Fuad Sparta (10 Points)**

Di sebuah laboratorium komputer, seorang mahasiswa bernama Fuad Sparta sedang menguji program C yang ia buat. Program tersebut menciptakan beberapa Child Process untuk menjalankan tugas tertentu. Namun terjadi secara terpisah dua kejadian aneh yang membingungkan mahasiswa tersebut:

KEJADIAN 1:

Salah satu Child Process telah menyelesaikan semua instruksinya dan berhenti. Namun, saat mahasiswa melihat daftar process menggunakan perintah <ps>, process tersebut masih ada di sana dengan status <defunct>. Process ini tidak menggunakan CPU, memori, ataupun resource lainnya, tetapi menempati slot di process table.

KEJADIAN 2:

Pada saat yang lain, parent process tiba-tiba mengalami crash dan berhenti secara mendadak saat child process lainnya masih aktif berjalan. Anehnya, child process yang ditinggalkan tidak ikut mati. Hal ini dikarenakan adanya mekanisme dalam OS yang bekerja untuk child process tersebut.

PERTANYAAN :

Berdasarkan kedua kejadian diatas, bantu Mas Fuad dalam mengidentifikasi masing-masing fenomena apa yang terjadi pada KEJADIAN 1 dan KEJADIAN 2, serta jelaskan secara teknis dan singkat mengapa masing-masing fenomena tersebut dapat terjadi? (Simpan dengan nama “Jawaban_NIMTanpaNol.txt”)

JAWABAN :

KEJADIAN 1 : <u>Zombie Process</u>

Fenomena ini dapat terjadi karena parent tidak memanggil wait(), sehingga status child yang sudah mati tidak diterima oleh parent

KEJADIAN 2 : <u>Orphan Process</u>

Fenomena ini dinamakan Orphan Process, dimana child masih berjalan walaupun parent sudah mati, nantinya child akan diadopsi oleh proses init (PID 1)

2. **STUDI KASUS : Alur Komando Jend. Rusdi, Jendral Rongawi Kuno (30 Points)**

Dalam sebuah simulasi sistem pertahanan, terdapat sebuah rantai komando yang ketat. Struktur ini dimulai dari seorang Jendral (Process A) sebagai pemegang kendali utama.

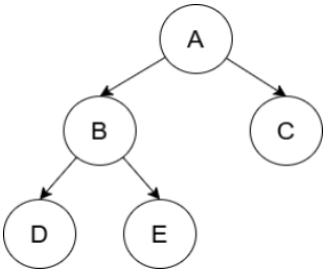
Berikut skenario kerja dari alur komando ini:

a. Jendral Rusdi (Process A) menerima sebuah pesan (input dari keyboard) berupa kalimat: “Quiz IF260L - Operating System”.
b. Jenderal (Process A) kemudian menurunkan perintah tersebut kepada dua bawahannya langsung yaitu Mayor Reza Kecap (Process B) dan Mayor Epstein (Process C).
c. Mayor Reza Kecap (Process B) memiliki kesibukan dengan lahan sawitnya, sehingga ia membagi tugas lagi kepada 2 sersan di bawah komandonya, yaitu Sersan Ambatukam (Process D) dan Sersan Ambatublow (Process E).
d. Mayor Epstein (Process C) bekerja secara mandiri untuk memproses pesan tersebut menjadi format rahasia (mengubah semua huruf menjadi HURUF KAPITAL).

PERTANYAAN: Executable dapat di download

Buatlah program dalam bahasa C menggunakan system call <fork()> untuk mensimulasikan hierarki alur komando di atas dengan ketentuan sebagai berikut:

a. Struktur Hierarki:
   i. Process A adalah Parent Process dari B dan C.
   ii. Process B adalah Parent Process dari D dan E.
b. Mekanisme Pesan: Kalimat yang diinput pada Process A harus dapat diakses oleh semua process turunannya (B, C, D, dan E).
c. Gunakan <wait()> atau <waitpid()> agar process tidak menjadi Zombie dan tampilan di layar muncul secara berurutan.
d. Urutan hierarki (A => B => D => E => C), dengan visual hierarki sebagai berikut: Simpan dengan nama “Soal1_NIMTanpaNol.c”



JAWABAN

soal1.c

3. **STUDI KASUS: Digitalisasi Kios Farhan Kebab (40 Points)**

Kios “Farhan Kebab” sedang melakukan digitalisasi sistem pemesanan mereka untuk menghindari kekacauan antara pelayan dan koki. Pemilik kios sekaligus koki ini yang bernama Farhan Kebab ingin sistem ini berjalan secara sinkron menggunakan prinsip Inter-Process Communication (IPC) pada Linux. Skenario sistem dirancang sebagai berikut:

a. Pelayan yaitu Si Imut (Program order.c) bertugas mencatat pesanan dari pelanggan. Setiap kali ada pesanan masuk, pelayan akan menambah jumlah antrian dalam “Kitab Catatan Digital” yang disimpan di memori bersama (Shared Memory).
b. Koki yaitu Farhan Kebab (Program cook.c) bertugas membaca “Kitab Catatan Digital” tersebut. Jika ada antrian, koki akan memasak pesanan satu per satu dan mengurangi jumlah antrian setelah masakkan selesai.
c. Karena adanya keterbatasan tempat “Kitab Catatan Digital” hanya mampu menampung 25 pesanan sekaligus. Jika sudah penuh, pelayan harus menunggu sampai koki menyelesaikan satu masakan.

PERTANYAAN: Executable dapat di download

Buat 2 program C yang memodelkan mekanisme diatas. Gunakan mekanisme Semaphore untuk mengatur akses ke Shared Memory agar tidak terjadi Race Condition saat pelayan menambah dan koki mengurangi pesanan secara bersamaan. Gunakan Shared Memory untuk menyimpan variabel globa yang bisa diakses oleh kedua program (misalnya: JUMLAH_ANTREAN). Karena kecepatan pelayanan mencatat dan koki memasak berbeda-beda, gunakan fungsi <sleep()> dengan argumen waktu acak (random) pada kedua program untuk mensimulasikan kondisi dunia nyata. Kedua program harus dapat dijalankan pada dua terminal terpisah.

INSTRUKSI DAN KETENTUAN TAMBAHAN:

a. Setiap kali terjadi perubahan, kedua terminal harus menampilkan informasi seperti “Status aksi (Sedang memesan atau Sedang Memasak)”, “Jumlah Pesanan baru masuk atau selesai”, dan “Total pesanan yang saat ini ada di dalam Kitab Catatan Digital”.
b. Buatlah 2 file .c dengan penamaan <order_NIMTanpaNol.c> dan <cook_NIMTanpaNol.c>. Ingat bahwa <order.c> harus berjalan dahulu sebelum <cook.c> dan jika <order.c> di-kill maka <cook.c> ikut ter-kill juga.

c. Pastikan kedua program menggunakan KeyID yang sama untuk mengakses Shared Memory dan Semaphore yang sama.

JAWABAN

order.c

cook.c

4. STUDI KASUS: Lomba Makan Mie Ayam Festival Kicau Mania (20 Points)

Di sebuah Festival Kicau Mania Jomokerto, diadakan sebuah lomba ikonik bernama ”Lomba Makan Mie Ayam”. Mas Azzril yang merupakan panitia divisi website dan sarana prasarana ingin mendigitalisasi sistem penjurian agar perhitungan jumlah cangkir yang diminum oleh setiap peserta akurat dan adil. Anda diminta untuk membantu Mas Azzril dalam membangun simulasi sistem ini menggunakan konsep Multi-Threading dan Synchronization pada bahasa C. Berikut spesifikasi teknis yang dibutuhkan:

a. Terdapat 3 orang kontestan tangguh (3 Threads) yang akan berlomba menghabiskan Mie Ayam dari sebuah meja besar yang sama. Setiap kontestan direpresentasikan oleh satu thread.
b. Satu thread khusus bertugas sebagai “Papan Skor” yang terus memantau dan menampilkan status sisa Mie Ayam di meja serta jumlah mangkuk yang sudah dikosongkan oleh masing-masing peserta secara real-time.
c. Thread utama bertugas menginisialisasi perlombaan, menerima input Jumlah Mie Ayam dan mengumumkan pemenang di akhir acara.

SPESIFIKASI TEKNIS:

a. Program menerima input jumlah total Mangkok Mie Ayam (n) dengan batasan 10 < n < 200.
b. Perlombaan harus diatur sedemikian rupa menggunakan MUTEX agar tidak ada satu pun thread kontestan yang memonopoli meja Mie Ayam (misalnya, satu kontestan menghabiskan 50 Mangkok Mie Ayam sekaligus tanpa memberi kesempatan kepada kontestan lain). TIPS: Gunakan <sleep(rand() % 5);>
c. Berikan informasi yang jelas setiap kali seorang kontestan berhasil mengambil Mie Ayam.
d. Setelah Mie Ayam di meja habis (n = 0), program harus menghentikan semua thread dengan benar dan menampilkan klasemen akhir (Juara 1, Juara 2, dan Juara 3) berdasarkan jumlah cangkir paling banyak dihabiskan.

PERTANYAAN: Kode disediakan pada lampiran.

Lengkapilah potongan kode yang ada pada lampiran pada bagian yang ditandai dengan comment. Pastikan logika sinkronisasi Anda tepat agar tidak terjadi race condition ataupun deadlock!

JAWABAN

soal4.c

5. **STUDI KASUS BONUS: 5 Filosof Yunani Kuno (20 Points)**

Di sebuah tempat pada zaman Yunani Kuno, terdapat 5 orang Filosof yang duduk melingkar di sebuah meja bundar untuk makan spaghetti. Di meja tersebut hanya tersedia 5 garpu yang diletakkan di antara setiap Filosof. Ada aturan makan yang menjadi budaya Yunani Kuno pada masa itu yaitu:

a. Untuk memakan spaghetti, seorang Filosof harus menggunakan 2 buah garpu (pada sebelah kiri dan sebelah kanan).
b. Jika garpu tidak tersedia, filosof tersebut harus menunggu sampai garpu diletakkan kembali oleh rekannya.
c. Setelah kenyang, seorang Filosof akan meletakkan kembali kedua sumpitnya dan lanjut merenung mengenai lahan sawit miliknya.

Adapun Pseudocode yang dapat membantu anda membayangkan masalah ini yaitu sebagai berikut:

```
do {
    // Mencoba mengambil garpu kiri
    wait( garpu[i] );

    // Mencoba mengambil garpu kanan
    wait( garpu[ (i + 1) % 5 ] );

    // --- MAKAN MIE AYAM ---

    // Meletakkan garpu kiri
    signal( garpu[i] );

    // Meletakkan garpu kanan
    signal( garpu[ (i + 1) % 5 ] );

    // --- BERPIKIR (THINK) ---

} while (TRUE);
```

PERTANYAAN BONUS:

a. Jelaskan skenario buruk atau masalah yang akan terjadi jika kelima Filosof tersebut merasa lapar secara bersamaan dan masing-masing berhasil mengambil garpu disebelah kirinya secara serentak! Sebutkan istilah teknis untuk kondisi ini. (5 Points)
b. Mengapa algoritma di atas dianggap melanggar prinsip Liveness? Apa yang akan terjadi pada sistem jika kondisi pada soal nomor a terjadi? Jelaskan secara singkat! (5 Points)
c. Berikan solusi agar perjamuan tersebut tetap berjalan dengan lancar tanpa ada guru besar yang kelaparan selamanya! Jelaskan secara singkat! (10 Points)

JAWABAN

a. Semua filosofis saling menunggu satu sama lain dalam lingkaran tertutup, tidak ada yang bisa maju, dan tidak ada yang melepas garpunya, istilah ini dinamakan **Deadlock.**
b. Algoritma di pseudocode melanggar prinsip **Liveness** karena tidak menjamin setiap filosof bisa makan, sistem bisa masuk ke state dimana semua thread terblokir selamanya.

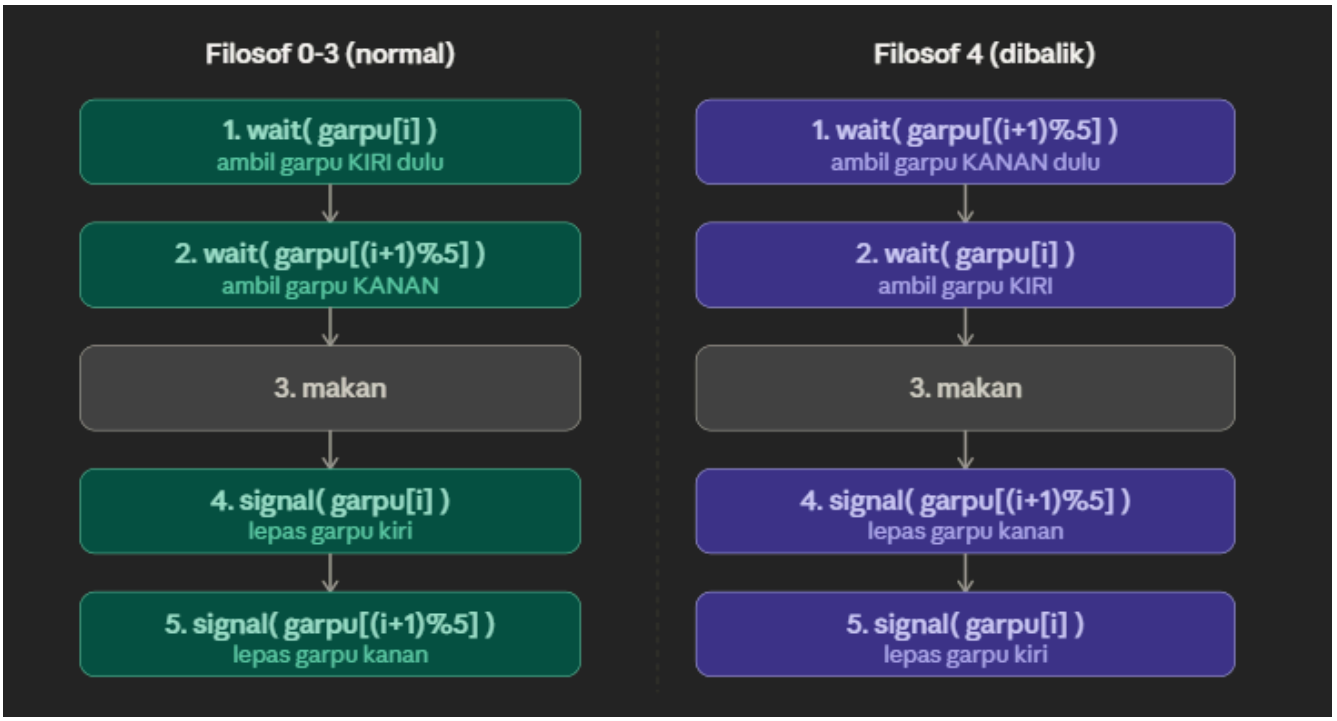
   Yang terjadi jika kondisi (a) terjadi:
   - Semua thread terjebak di wait(garpu_kanan) selamanya
   - Tidak ada thread yang bisa melanjutkan eksekusi
   - Sistem berhenti total
   - Satu-satunya cara keluar adalah kill program secara manual

   Ini melanggar 2 properti liveness:
   - Progress : tidak ada thread yang bisa masuk critical section
   - Starvation-freedom : semua filosof kelaparan selamanya

c. Ada 3 solusi untuk permasalahan ini

- **Solusi 1 - resource ordering (hierarki garpu)**

  Balik urutan pengambilan garpu untuk satu filosof saja (filosof 4)



- **Solusi 2 - batasi jumlah filosof yang boleh makan sekaligus**

  Menambahkan semaphore **meja** dengan nilai 4, maksimal hanya 4 filosof yang boleh mencoba garpu secara bersamaan

  wait(meja); // masuk "antrian meja"
  wait(garpu[i]);
  wait(garpu[(i+1)%5]);
  // makan
  signal(garpu[i]);
  signal(garpu[(i+1)%5]);
  signal(meja); // keluar "antrian meja"

- **Solusi 3 - Atomic pickup (ambil 2 garpu sekaligus atau tidak sama sekali)**

  Gunakan mutex global untuk melindungi proses pengambilan kedua garpu, filosof hanya boleh mengambil garpu jika kedua garpu tersedia, jika tidak, ia tidak mengambil satupun dan harus menunggu